Step 1 — The Code Index and the Context Builder — Solution

> Back to the step · Hint 1 · Hint 2

Story: this builds the code store, the retriever, and the deterministic context and citation builders — the parts that turn raw retrieved code snippets into a model-ready text block and a clean citation list, with no model call involved at all. **If not:** grounding and citations are the whole value of this project; debugging them *and* real model behavior at the same time in Step 3 would be miserable.

Setup files

`requirements.txt`: `langchain-openai`, `langchain-community`, `langchain-text-splitters`, `chromadb`, `openai`, `python-dotenv`. `.env.example`: `OPENAI_API_KEY=sk-your-key-here` / `LOG_LEVEL=INFO`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`, `chroma_db/`. Every folder under `src/` gets an empty `__init__.py`. `LICENSE` is the MIT text.

Settings, errors, logging, model

```
# src/exceptions.py
class MissingConfigError(Exception):
    """A required setting is missing from the environment."""
    pass
```

```
# src/config.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError

@dataclass
class Config:
    openai_api_key: str
    log_level: str
    repo_dir: str

def require_env(key):
```

```
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config():
    load_dotenv()
    return Config(
        openai_api_key=require_env("OPENAI_API_KEY"),
        log_level=os.getenv("LOG_LEVEL", "INFO"),
        repo_dir=os.getenv("REPO_DIR", "sample_repo"),
    )
```

```
# src/utils/logger.py
import logging
import os

def get_logger(name):
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler()
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

```
# src/models/llm.py
from langchain_openai import ChatOpenAI

def get_model(config):
    return ChatOpenAI(
        model="gpt-4o-mini", temperature=0, api_key=config.openai_api_key
    )
```

The code index

```
# src/rag/code_store.py
# Load the repo's .py files, chunk them, keep each chunk's file path in its
# metadata (that path is the citation), embed, and store in Chroma.
from langchain_community.document_loaders import DirectoryLoader, TextLoader
from langchain_openai import OpenAIEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter

import chromadb

COLLECTION_NAME = "code"
EMBEDDINGS = OpenAIEmbeddings(model="text-embedding-3-small")

def build_code_store(repo_dir):
    loader = DirectoryLoader(repo_dir, glob="**/*.py", loader_cls=TextLoader)
    documents = loader.load()
    splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=60)
    chunks = splitter.split_documents(documents)

    client = chromadb.Client()
    try:
        client.delete_collection(COLLECTION_NAME)
    except Exception:
        pass
    collection = client.create_collection(COLLECTION_NAME)

    ids = []
    texts = []
    metadatas = []
    for index in range(len(chunks)):
        ids.append(f"chunk-{index}")
        texts.append(chunks[index].page_content)
        path = chunks[index].metadata.get("source", "unknown")
        metadatas.append({"file": path.split("/")[-1]})
    collection.add(ids=ids, documents=texts, metadatas=metadatas,
                   embeddings=EMBEDDINGS.embed_documents(texts))
    return collection
```

```
# src/rag/retriever.py
# Return the code snippets closest to a question, each with its file path.
from src.rag.code_store import EMBEDDINGS

def retrieve_snippets(collection, question, k=4):
    if collection.count() == 0:
        return []
    results = collection.query(
        query_embeddings=[EMBEDDINGS.embed_query(question)], n_results=k,
```

```
    )
    documents = results["documents"][0]
    metadatas = results["metadatas"][0]
    snippets = []
    for index in range(len(documents)):
        snippets.append({
            "code": documents[index],
            "file": metadatas[index].get("file", "unknown"),
        })
    return snippets
```

The deterministic context and citation builders

```
# src/context.py
# Turn retrieved snippets into one model-ready block (each under a # File:
# header) and the de-duplicated list of files to cite. Plain Python, so it's
# fully testable with hand-made snippets - no model or embeddings.
def build_context(snippets):
    blocks = []
    for snippet in snippets:
        blocks.append(f"# File: {snippet['file']}\n{snippet['code']}")
    return "\n\n".join(blocks)

def cited_files(snippets):
    files = []
    for snippet in snippets:
        if snippet["file"] not in files:
            files.append(snippet["file"])
    return files
```

The tests (no model, no embeddings)

```
# tests/unit/test_config.py
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "REPOSAGE_TEST_SETTING"

def test_missing_setting():
```

```
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")

def test_config_loads():
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 2 (config loads)    -> {result}")

if __name__ == "__main__":
    test_missing_setting()
    test_config_loads()
```

```
# tests/unit/test_context.py
# Run from the repo root: python -m tests.unit.test_context
from src.context import build_context, cited_files

SNIPPETS = [
    {"file": "auth.py", "code": "def login(user, pw):\n    return check(pw)"},
    {"file": "db.py", "code": "def connect():\n    return pool.get()"},
    {"file": "auth.py", "code": "def logout(user):\n    session.clear()"},
]

def test_every_snippet_has_a_file_header():
    context = build_context(SNIPPETS)
    result = "OK"
    if context.count("# File:") != 3:
        result = "WRONG"
    if "# File: auth.py" not in context:
        result = "WRONG"
    print(f"case 1 (file headers in context) -> {result}")

def test_cited_files_are_deduped():
    files = cited_files(SNIPPETS)
    result = "OK"
    if files != ["auth.py", "db.py"]:
        result = "WRONG"
    print(f"case 2 (cited files deduped)     -> {result}")
```

```
if __name__ == "__main__":
    test_every_snippet_has_a_file_header()
    test_cited_files_are_deduped()
```

Expected output:

```
case 1 (missing setting) -> OK
case 2 (config loads)    -> OK
```

```
case 1 (file headers in context) -> OK
case 2 (cited files deduped)     -> OK
```

The context and citations are proven with hand-made snippets — so Step 3 only has to feed in real retrieval results, not re-test the formatting.